# Code Walkthrough & Tech Stack

Geospatial Flood-Risk Modelling (Climate Data + HMM), Kigali

Revision companion: how the product is built, function by function

**Kellen Murerwa · Supervisor: Emmanuel Adjei · ALU BSc SE**

# 1 · Pipeline orchestrator — main.py

```
STAGES = {
  "build":    "build_real_dataset.py",  # real data -> parquet
  "polygons": "add_official_polygons.py",# official MOE polygons
  "train":    "train_real.py",          # models + HMM (2024 hold-out)
  "evaluate": "evaluate_spatial.py",    # spatial validation + map
}

# one command runs the whole thing, reproducibly:
#   python main.py all        build -> train -> evaluate
#   python main.py dashboard  launch the Streamlit app

if args.stage == "all":
    for s in ["build", "train", "evaluate"]:
        run_module(STAGES[s])
```

- Single CLI entry point; each stage is an independent, re-runnable module.
- `all` chains build → train → evaluate; `dashboard` serves the app.
- API responses cached in data_cache/, so re-runs are fast & offline.
- This is the slide to open on — it frames the whole architecture.

# 2 · Data acquisition — real, free, keyless

```
# rainfall: Open-Meteo ERA5 reanalysis archive
requests.get("https://archive-api.open-meteo.com/v1/archive",
  params={"latitude": la, "longitude": lo,
          "start_date": "2018-01-01", "end_date": "2024-12-31",
          "daily": "precipitation_sum"})

# OpenStreetMap via Overpass (rotates mirrors + User-Agent)
requests.post(url, data={"data": query}, headers=OVERPASS_HEADERS)

# official flood zones: Rwanda GeoPortal (ArcGIS REST)
requests.get(SERVICE, params={"where": "objectid IN (1,2,3)",
                              "outSR": "4326", "f": "geojson"})
```

- Four open sources, no API keys: ERA5 rainfall, SRTM elevation, OSM exposure, MOE flood polygons.
- Overpass call hardened: User-Agent + mirror rotation + JSON validation.
- Honest caveat: rainfall is ERA5 reanalysis (substituted for Meteo Rwanda).

# 3 · Feature engineering

```
# rainfall accumulation windows (rolling sums)
"rainfall_3d_mm":  s.rolling(3,  min_periods=1).sum()
"rainfall_7d_mm":  s.rolling(7,  min_periods=1).sum()
"rainfall_14d_mm": s.rolling(14, min_periods=1).sum()

# distance to nearest river: spatial KD-tree (fast NN)
rtree = cKDTree(river_xy)
dist, _ = rtree.query(cell_xy)

# slope from the elevation grid's gradient
gy, gx = np.gradient(Z, CELL_M, CELL_M)
slope  = np.degrees(np.arctan(np.hypot(gx, gy)))

# road length density & building count density per km^2
```

- Turns raw data into flood-relevant signal: multi-day rain build-up, terrain, proximity & exposure.
- `cKDTree` (SciPy) = efficient nearest-river distance for 729 cells.
- Slope derived from the SRTM elevation surface via numpy gradient.
- SHAP later confirms the rainfall-accumulation features dominate.

# 4 · Flood-pressure labels

```
# rainfall trigger (saturating) x static susceptibility
trig = 0.6*np.tanh(rain3/60) + 0.4*np.tanh(rain7/110)
base = trig * (0.12 + 0.88*susc)     # rain GATED by terrain

# + UNOBSERVED drainage latent + daily noise
#   (deliberately NOT given to the model as features)
score = base + cell_latent + daily_noise

# 50 / 30 / 20  ->  Low / Moderate / High
q1, q2 = np.quantile(score, [0.50, 0.80])
state = np.where(score <= q1, "Low",
        np.where(score <= q2, "Moderate", "High"))
```

- Label = rainfall trigger × terrain susceptibility (heavy rain on high, well-drained ground stays Low).
- Latent drainage + noise are hidden from the model on purpose — keeps the task realistic and NON-circular.
- Expect this question: 'are your labels derived?' Yes — and the validity limit is owned in the report.

# 5 · Models & temporal train/val/test split — train_real.py

```
tr = df[df.date.dt.year <= 2022]   # train  2018-2022
va = df[df.date.dt.year == 2023]   # validate 2023 (over-fit check)
te = df[df.date.dt.year == 2024]   # test on UNSEEN 2024

models = {
 "LogisticRegression": make_pipeline(StandardScaler(),
                       LogisticRegression(max_iter=2000)),
 "DecisionTree": DecisionTreeClassifier(max_depth=8),
 "RandomForest": RandomForestClassifier(n_estimators=300,
                 min_samples_leaf=5, max_leaf_nodes=4096, n_jobs=2),
 "XGBoost": xgb.XGBClassifier(n_estimators=400, max_depth=5,
            learning_rate=0.07, tree_method="hist",
            objective="multi:softprob", num_class=3),
}
# + rainfall-threshold & static-polygon baselines
```

- 3-way temporal split (not random) = the honest test; 2024 is never seen in training.
- Train ≈ validation ≈ test macro-F1 → no over-fitting (reported per split).
- scikit-learn (LR/DT/RF) + XGBoost; hyper-params are explicit, not defaults.
- Result: XGBoost (deployed) 0.813 / 0.843; RF 0.813 / 0.849.

# 6 · HMM temporal layer

```
from hmmlearn import hmm

# per-cell daily rainfall sequences -> 3 latent states
model = hmm.GaussianHMM(n_components=3,
            covariance_type="diag", n_iter=200)
model.fit(Xs, lengths)

decoded = model.predict(Xs, [L])   # Viterbi state per day
T = model.transmat_                # daily transition matrix
#   High->High = 0.91  (flood pressure is persistent)

# next-step accuracy: argmax(T[state_t]) vs truth_{t+1}
```

- Models day-to-day DYNAMICS of flood pressure (not just a static snapshot).
- Transition matrix shows strong persistence & neighbour-only moves.
- Dashboard renders a PER-CELL transition matrix — P(state tomorrow | today).
- Honest framing: HMM is explanatory; the supervised layer is the predictor (next-step acc 0.45).

# 7 · Evaluation & targets

```
macro_f1    = f1_score(yte, p, average="macro")
high_recall = recall_score(yte, p, labels=[2], average="macro")

# spatial validation vs OFFICIAL polygons (evaluate_spatial.py)
lift = P(official_zone | predicted_High) / P(official_zone)
odds = high_rate_inside_zone / high_rate_outside_zone

targets_met = {
  "macro_f1>=0.75":   True,    # 0.813
  "high_recall>=0.80": True,   # 0.843
}   # enrichment 1.60x (>=1.4), odds 1.85x (>=1.5)
```

- Reports F1, accuracy, recall + a spatial check against the official map.
- Enrichment & odds-ratio chosen because polygons cover only ~19% of cells.
- All proposal targets met; results written to results_summary.json.

# 8 · The product — dashboard.py

```
import streamlit as st, pydeck as pdk

@st.cache_resource
def load_model():
    b = joblib.load("model_outputs_real/xgboost_model.joblib")
    return b["model"], b["features"]

day["pred_i"] = model.predict(day[feat].values)   # per cell
proba = model.predict_proba(day[feat].values)     # confidence

pdk.Layer("ScatterplotLayer", day,        # the live map
    get_fill_color="color", get_radius=110, pickable=True)
```

- Streamlit app: pick a date → live Low/Mod/High map over 729 cells.
- `@st.cache_resource` loads the model once; `pydeck` renders the map.
- Click a cell → features + class probabilities; p95 query ≈ 9.6 ms.
- Demo this live, or play flood_pressure_animation.gif.

# 9 · Technologies, languages & ML tools

| | |
|---|---|
| **Language** | Python 3.11 |
| **Data handling** | pandas · numpy · pyarrow (Parquet) · SciPy (cKDTree) |
| **Machine learning** | scikit-learn (LogReg, DecisionTree, RandomForest, metrics, scaling) · XGBoost · hmmlearn (HMM) · SHAP |
| **Data sources / APIs** | Open-Meteo (ERA5 rainfall, SRTM elevation) · OpenStreetMap / Overpass · Rwanda GeoPortal (ArcGIS REST) |
| **Visualisation / app** | matplotlib · Streamlit · pydeck |
| **Persistence / infra** | joblib (model artefacts) · Docker · GitHub · Streamlit Cloud / Render |
| **Testing / quality** | pytest (10 tests) · temporal hold-out · spatial validation |

BE READY FOR

# 10 · What to explain (defense cheat-sheet)

- Architecture: one pipeline, four stages (main.py) — start here.
- Data provenance: the four open sources + why ERA5 instead of Meteo Rwanda.
- Feature engineering: rainfall accumulation windows, distance-to-river (cKDTree), slope from elevation gradient, densities.
- Labels: rainfall trigger × susceptibility + hidden latent/noise — why it's NOT circular (the most likely hard question).
- Why a temporal split (train 2018–22, validate 2023, test 2024) rather than a random split.
- Model choice: RF/XGBoost vs baselines; what the numbers mean (macro-F1, High-recall) and why recall is weighted.
- Spatial validation: enrichment 1.60× & odds 1.85× — why not raw containment (polygons cover only ~19%).
- HMM: what it adds (dynamics/persistence) and its honest limit vs supervised.
- The dashboard + tests + deployment = a usable, validated artefact, not just a table.